

Your University Name

Department of Computer Science / Information Technology

Library Management System

A Web Application for Library Operations

Project Report

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology / Bachelor of Engineering

Submitted by
Pratham Kashyap
Registration No.

Under the guidance of
Guide Name
Designation

Academic Year 2025–2026

Certificate

This is to certify that the project entitled “**Library Management System**”, submitted by **Pratham Kashyap** (Registration No.: _____) to the Department of _____, _____ University, in partial fulfillment of the requirements for the award of the degree of _____, is a bonafide record of the work carried out under my guidance and supervision.

Date: _____

Place: _____

Guide Name

Designation

Department

Head of Department

Name

Department

Declaration

I hereby declare that the project entitled “**Library Management System**” submitted to _____ University is a record of original work done by me under the guidance of _____, and this project has not been submitted for the award of any other degree, diploma, or fellowship.

Pratham Kashyap

Registration No.

Date: _____

Place: _____

Acknowledgements

I would like to express my sincere gratitude to my project guide, _____, for their valuable guidance and constant encouragement throughout the development of this project. I am also thankful to the Head of Department, _____, for providing the necessary facilities and support.

I would also like to thank my family and friends for their continuous support and motivation.

Abstract

This project presents the design and implementation of a Library Management System — a full-stack web application built to automate core library operations. The system handles book catalogue management, student and librarian account administration, and the complete borrow/return lifecycle with real-time availability tracking.

The backend is developed using Spring Boot 3.5 with Java 21, employing a layered MVC architecture with Spring Data JPA for persistence and Spring Security for authentication and authorization. The frontend is a responsive single-page application built with vanilla JavaScript, HTML, and CSS, communicating with the backend through a RESTful API.

The system enforces role-based access control across three user roles (Administrator, Librarian, Student), implements server-side and client-side validation, and provides CSRF protection for secure form submissions. The project includes 24 REST endpoints, 5 database tables, and a suite of integration tests covering authentication, CRUD operations, and business rule enforcement.

Keywords: Library Management, Spring Boot, REST API, Spring Security, Single-Page Application, Role-Based Access Control.

Table of Contents

Contents

1	Introduction	4
1.1	Background	4
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Scope and Limitations	4
2	Requirements Analysis	5
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	5
2.3	Use Cases	5
3	System Architecture	6
3.1	Technology Stack	6
3.2	Layered Architecture	6
3.3	Package Structure	7
3.4	Project Structure	7
4	Database Design	9
4.1	Entity-Relationship Diagram	9
4.2	Table Relationships	10
4.3	Design Decisions	10
5	Implementation	11
5.1	Security Implementation	11
5.1.1	Authentication	11
5.1.2	CSRF Protection	11
5.1.3	Authorization	11
5.2	Backend Implementation	12
5.2.1	Controllers	12
5.2.2	Services and Business Rules	12
5.2.3	DTOs and Validation	12
5.2.4	Exception Handling	13
5.3	Frontend Implementation	13
5.3.1	SPA Architecture	13
5.3.2	CSS Architecture	13
5.3.3	JavaScript Modules	13
5.4	API Reference	14
6	Application Workflows	15
6.1	Authentication Flow	15
6.2	Borrow Workflow	15
6.3	Return Workflow	15
7	Screenshots	16
7.1	Authentication	16
7.2	Dashboard	16
7.3	Book Management	17
7.4	Student and Librarian Management	17
7.5	Borrow Records	18
7.6	Profile	19
7.7	Validation and Error Handling	19

7.8 Mobile Responsive Design	20
8 Testing	21
8.1 Test Infrastructure	21
8.2 Test Coverage	21
8.3 Key Test Scenarios	21
8.4 Test Execution	21
9 Future Scope	22
10 Conclusion	23
11 References	24

1 Introduction

1.1 Background

Libraries are fundamental to educational institutions, providing access to knowledge resources for students and faculty. Managing a library's inventory, tracking borrowed materials, and maintaining borrower records is a complex operational task that benefits significantly from automation.

Traditional manual approaches to library management — paper-based registers, card catalogues, and physical sign-out sheets — are prone to errors, difficult to search, and impossible to scale. A digital library management system addresses these challenges by providing a centralized, secure, and searchable platform for all library operations.

1.2 Problem Statement

The project addresses the need for a web-based library management system that:

- Maintains an accurate, searchable catalogue of books with real-time availability status.
- Tracks the complete lifecycle of book loans — from borrowing to return — with borrower records preserved for audit.
- Manages student and librarian accounts with appropriate access restrictions.
- Provides a responsive interface accessible on desktop and mobile devices.
- Enforces data integrity through server-side validation and transactional business rules.

1.3 Objectives

1. Design and implement a layered MVC architecture using Spring Boot and Spring Security.
2. Build a responsive single-page application (SPA) with role-based navigation.
3. Implement CRUD operations for books, students, and librarians with server-side validation.
4. Develop a borrow/return workflow that atomically manages book availability.
5. Enforce role-based access control with three distinct user roles.
6. Protect the application with session-based authentication and CSRF protection.
7. Write integration tests to verify core functionality and security behavior.

1.4 Scope and Limitations

The v1.0.0 release focuses on core library operations. The following features are intentionally deferred for future development: book categories, pagination, advanced search, fines, notifications, Docker deployment, Swagger/OpenAPI, JWT authentication, reservations, and physical-copy modelling.

2 Requirements Analysis

2.1 Functional Requirements

ID	Requirement	Status
FR-01	Users can log in, log out, and view current session information	Implemented
FR-02	Admins and librarians can view dashboard with aggregate statistics	Implemented
FR-03	Authorized staff can create, list, search, update, and delete books	Implemented
FR-04	Admins and librarians can manage student accounts and profiles	Implemented
FR-05	Only admins can manage librarian accounts and profiles	Implemented
FR-06	Staff can borrow an available book, creating a record and marking the book unavailable	Implemented
FR-07	Staff can return an active borrow record, restoring book availability	Implemented
FR-08	The system rejects unavailable-book borrows and duplicate returns with clear errors	Implemented
FR-09	Authenticated users can view their own profile information	Implemented
FR-10	The frontend is responsive and works on desktop and mobile	Implemented

Table 1: Functional requirements and implementation status.

2.2 Non-Functional Requirements

ID	Requirement
NFR-01	Maintain a layered Browser → Controller → Service → Repository → MySQL architecture
NFR-02	Never expose JPA entities or password hashes through the API
NFR-03	Store only BCrypt password hashes and enforce server-side role checks
NFR-04	Preserve borrow history and prevent destructive cascades from removing audit data
NFR-05	Return documented HTTP status codes and uniform JSON error responses
NFR-06	Responsive, keyboard-accessible, readable UI across viewports

Table 2: Non-functional requirements.

2.3 Use Cases

The system supports three actors with distinct use cases:

Actor	Use Cases
Administrator	Log in/out · View dashboard · Manage books (CRUD + search) · Manage students (CRUD) · Manage librarians (CRUD) · Borrow/return books · View profile
Librarian	Log in/out · View dashboard · Manage books (CRUD + search) · Manage students (CRUD) · Borrow/return books · View profile
Student	Log in/out · Browse books · View profile

Table 3: Use cases by actor role.

3 System Architecture

3.1 Technology Stack

Layer	Technology	Version
Backend runtime	Java (OpenJDK)	21
Application framework	Spring Boot	3.5.0
Security	Spring Security	6.5
Persistence	Spring Data JPA + Hibernate	Managed by Boot
Validation	Jakarta Bean Validation (Hibernate Validator)	Managed by Boot
Production database	MySQL	8+
Test database	H2 (MySQL compatibility mode)	Managed by Boot
Build tool	Apache Maven (with wrapper)	Wrapper-managed
Test framework	JUnit 5 + MockMvc + Spring Security Test	Managed by Boot
Frontend	Vanilla HTML / CSS / JavaScript (ES Modules)	—
Typography	Google Fonts (Inter, Poppins)	CDN

Table 4: Technology stack.

3.2 Layered Architecture

The application follows a strict four-layer MVC architecture within a single deployable Spring Boot monolith. Each layer communicates only with its immediate neighbor, enforcing separation of concerns.

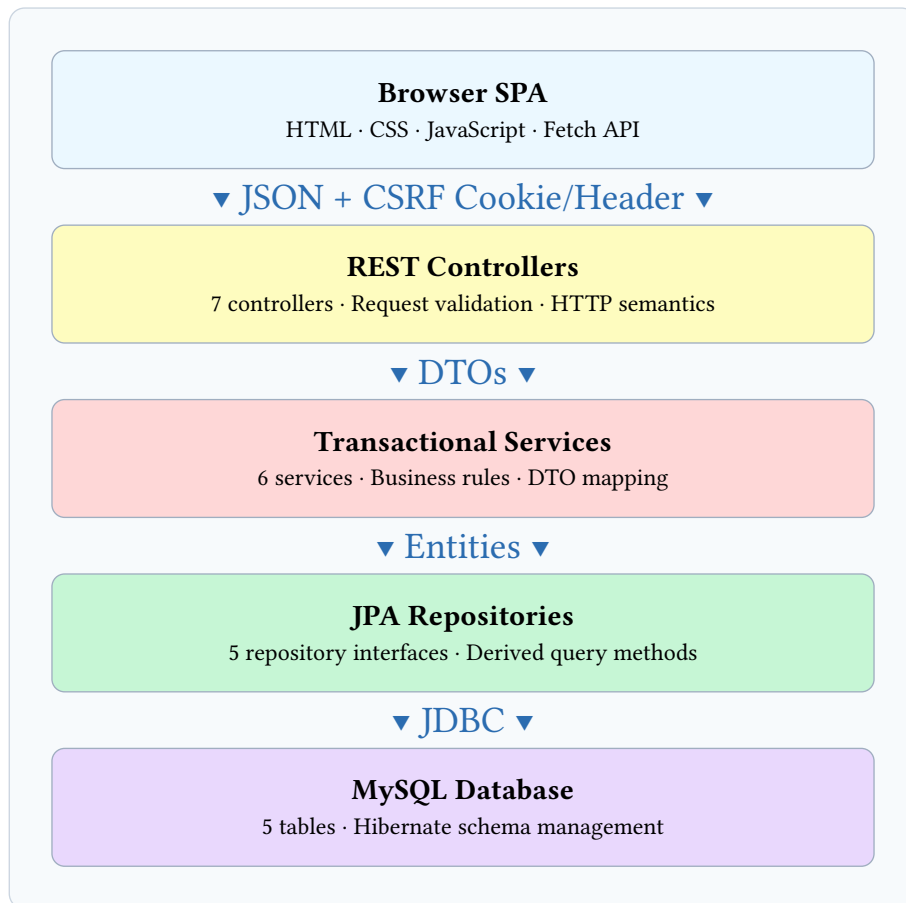


Figure 1: Layered architecture — each layer communicates only with its immediate neighbor.

3.3 Package Structure

Package	Classes	Responsibility
com.example.lms	1	Application entry point and admin seed
config	1	BCrypt PasswordEncoder bean
controller	7	REST endpoints and request validation
dto	14	Immutable Java records for request/response transfer
entity	7	JPA entities, mapped superclass, role enum
exception	4	Domain exceptions and centralized error handler
repository	5	JPA repository interfaces
security	5	Spring Security configuration and handlers
service	6	Transactional business logic

Table 5: Package structure with class counts.

3.4 Project Structure

```

Library Management System/
├── backend/
│   ├── pom.xml
│   ├── src/main/java/.../lms/
│   │   ├── controller/
│   │   └── dto/
│   └── Build configuration
│       ├── 7 REST controllers
│       └── 14 Java record DTOs

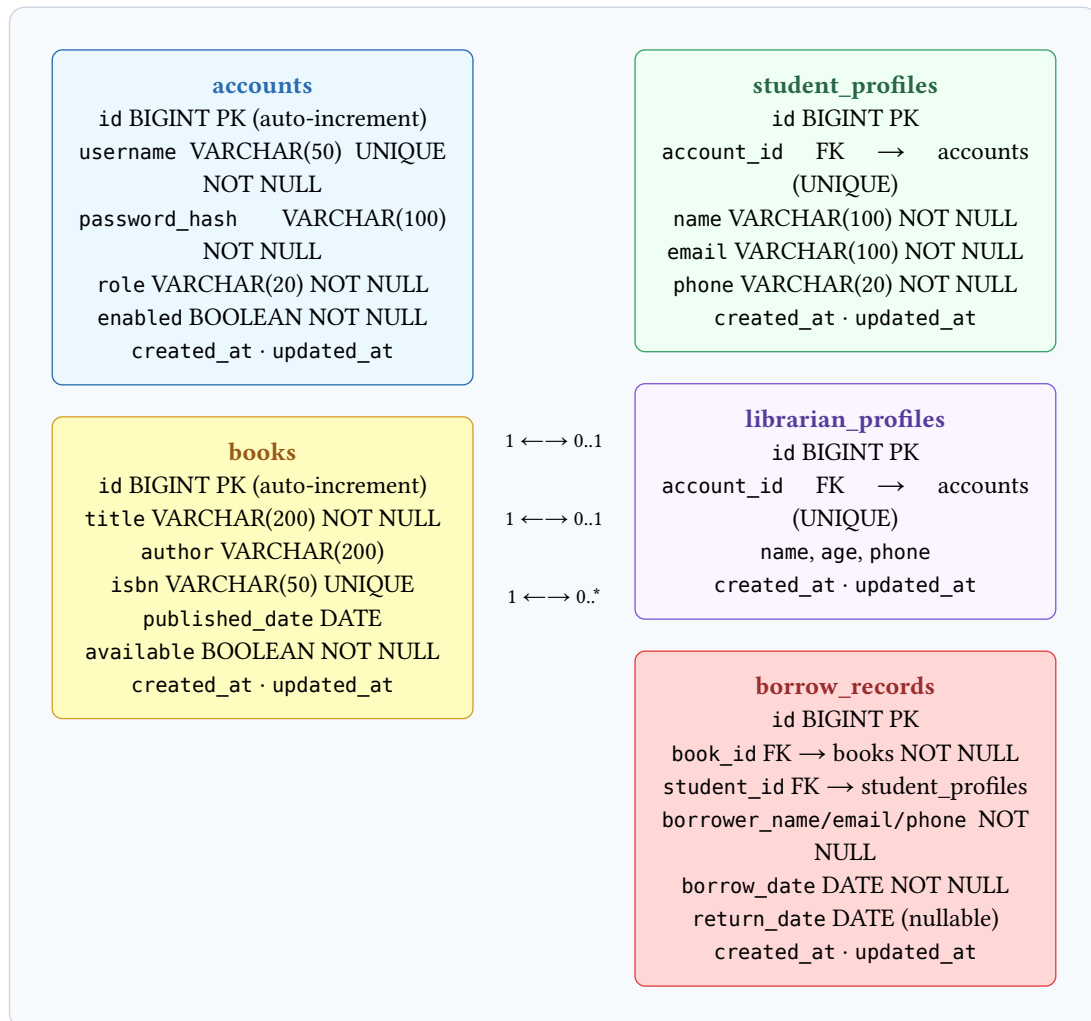
```

		— entity/	JPA entities + Role enum
		— exception/	Custom exceptions + handler
		— repository/	JPA repository interfaces
		— security/	Security configuration
		— service/	Transactional services
		— src/main/resources/	
		— application.properties	
		— static/	SPA frontend
		— index.html	7-page SPA shell
		— css/	Design tokens, layout, components
		— js/	SPA orchestrator + API clients
		— components/	Reusable modal component
		— docs/	Project documentation
		— screenshots/	Desktop and mobile evidence

4 Database Design

The database consists of five tables managed by Hibernate's automatic schema generation from JPA entity annotations. All entities inherit audit fields (`created_at`, `updated_at`) from a common `AuditableEntity` mapped superclass.

4.1 Entity-Relationship Diagram



Listing 1: Entity-relationship diagram.

4.2 Table Relationships

From	Cardinality	To	Join Column
accounts	1 : 0..1	student_profiles	account_id (UNIQUE, NOT NULL)
accounts	1 : 0..1	librarian_profiles	account_id (UNIQUE, NOT NULL)
books	1 : 0..*	borrow_records	book_id (NOT NULL)
student_profiles	1 : 0..*	borrow_records	student_id (nullable)

Table 6: Entity relationships with join columns.

4.3 Design Decisions

- **Account-Profile separation:** Authentication credentials are stored in accounts, while role-specific data (email for students, age for librarians) is stored in separate profile tables. This allows shared login logic with distinct profile schemas.
- **Borrower snapshotting:** The borrow_records table stores a snapshot of the borrower's name, email, and phone at borrow time, ensuring the historical record remains accurate even if the student's profile is later updated.
- **Availability tracking:** A boolean available flag on the books table is toggled atomically within the same transaction as borrow/return operations, preventing race conditions.

5 Implementation

5.1 Security Implementation

Security was implemented using Spring Security 6.5 with the following components:

5.1.1 Authentication

The authentication pipeline uses `BCryptPasswordEncoder` for password hashing, `AccountUserDetailsService` for loading user credentials from the database, and `DaoAuthenticationProvider` to tie them together. On successful login, a `SecurityContext` is created and stored in an HTTP session.

```
// AuthService.java – Login logic
public AuthenticatedUserResponse login(LoginRequest request,
                                      HttpSession session) {
    var auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            request.username(), request.password()));
    var ctx = SecurityContextHolder.createEmptyContext();
    ctx.setAuthentication(auth);
    SecurityContextHolder.setContext(ctx);
    session.setAttribute(SECURITY_CONTEXT_KEY, ctx);
    // ... return user response
}
```

A seed admin account with a BCrypt-hashed password is created on first startup via a `CommandLineRunner` in the application class.

5.1.2 CSRF Protection

The application uses a SPA-compatible CSRF scheme. The server issues a readable XSRF-TOKEN cookie, and the JavaScript frontend reads this cookie and sends it back as an X-XSRF-TOKEN header on state-changing requests. A custom `SpaCsrfTokenRequestHandler` was implemented to handle the token validation correctly under Spring Security 6.5, which defaults to XOR-encoded tokens that a JavaScript client cannot produce from a cookie value.

5.1.3 Authorization

Role-based access is enforced through URL-pattern matching in the `SecurityFilterChain`. The rules are evaluated in declaration order:

Resource	Admin	Librarian	Student	Public
Login, CSRF, static assets	✓	✓	✓	✓
Books (read only)	✓	✓	✓	—
Books (create, update, delete)	✓	✓	—	—
Students (all operations)	✓	✓	—	—
Borrow records, Dashboard	✓	✓	—	—
Librarians (all operations)	✓	—	—	—
Profile	✓	✓	✓	—

Table 7: Role-permission matrix.

Unauthorized requests receive JSON error responses — 401 Unauthorized for unauthenticated users (via `RestAuthenticationEntryPoint`) and 403 Forbidden for insufficient roles (via `RestAccessDeniedHandler`).

5.2 Backend Implementation

5.2.1 Controllers

All 7 controllers follow a consistent pattern: constructor injection of their service, thin delegation with no business logic, and `@Valid` annotation on request bodies.

Controller	Endpoints	Responsibility
<code>AuthController</code>	4	CSRF bootstrap, login, logout, current user
<code>BookController</code>	5	Book CRUD with search
<code>BorrowRecordController</code>	3	Borrow, return, list records
<code>DashboardController</code>	1	Aggregate statistics
<code>LibrarianController</code>	5	Librarian CRUD (admin-only)
<code>ProfileController</code>	1	Current user profile
<code>StudentController</code>	5	Student CRUD

Table 8: Controller summary.

5.2.2 Services and Business Rules

Services contain all business logic and act as the transactional boundary. Key business rules enforced at the service layer:

- **ISBN uniqueness:** Checked before save and also caught as a database constraint violation fallback.
- **Username uniqueness:** Verified before creating student or librarian accounts.
- **Book availability:** Only available books can be borrowed; the flag is toggled atomically within the transaction.
- **Return validation:** Already-returned records cannot be returned again.
- **Deletion protection:** Books with borrow history cannot be deleted (preserves audit trail).

5.2.3 DTOs and Validation

All 14 DTOs are Java record types, providing immutable data transfer. Request DTOs carry Jakarta Bean Validation annotations:

DTO	Field	Constraints
BookRequest	title	@NotBlank @Size(max=200)
StudentRequest	password	@NotBlank @Size(min=8, max=100)
	email	@NotBlank @Email @Size(max=100)
LibrarianRequest	age	@Min(18) @Max(100)
BorrowRequest	bookId	@NotNull
LoginRequest	username	@NotBlank

Table 9: Key validation constraints (abbreviated).

Separate `*UpdateRequest` DTOs exist for students and librarians, which include profile fields but omit the password to prevent accidental credential resets during profile updates.

5.2.4 Exception Handling

A centralized `GlobalExceptionHandler` (`@RestControllerAdvice`) maps domain exceptions to a uniform `ApiErrorResponse` JSON structure:

Exception	HTTP Status	Error Code
<code>ResourceNotFoundException</code>	404	NOT_FOUND
<code>ConflictException</code>	409	CONFLICT
<code>BusinessRuleException</code>	400	Dynamic (e.g. BOOK_UNAVAILABLE)
<code>MethodArgumentNotValidException</code>	400	VALIDATION_ERROR + field errors

Table 10: Exception-to-HTTP mapping.

5.3 Frontend Implementation

5.3.1 SPA Architecture

The frontend is a single-page application contained in one `index.html` file with 7 `<section>` elements, each representing a page. Navigation uses `location.hash` to toggle page visibility without a full page reload. No frontend framework or build tooling is used — the application is served directly as static files.

5.3.2 CSS Architecture

Five CSS files implement a layered design system:

1. **tokens.css** — CSS custom properties for colors, spacing, and typography.
2. **base.css** — Resets and body typography.
3. **layout.css** — App shell grid, header, and navigation.
4. **components.css** — Cards, tables, buttons, badges, modals, and toasts.
5. **responsive.css** — Mobile breakpoint overrides for responsive layout.

The design uses an indigo/teal color scheme with Inter (body) and Poppins (headings) typefaces.

5.3.3 JavaScript Modules

The JavaScript is organized into ES modules:

- **main.js** — SPA orchestrator handling routing, rendering, and event wiring.
- **http.js** — Centralized Fetch wrapper handling CSRF headers, credentials, and error parsing.
- **6 API modules** — Thin wrappers for each backend resource (auth, books, borrow, dashboard, librarians, students).

- **modal.js** — Reusable form modal component with 4 form definitions and client-side + server-side validation rendering.

An `esc()` function escapes HTML special characters in all rendered user content to prevent XSS.

5.4 API Reference

#	Method	URL	Auth	Purpose
1	GET	/api/auth/csrf	Public	Bootstrap CSRF cookie
2	POST	/api/auth/login	Public	Authenticate and create session
3	POST	/api/auth/logout	Any	Invalidate session
4	GET	/api/auth/me	Any	Current session identity
5	GET	/api/books	Any	List/search books
6	GET	/api/books/{id}	Any	Get single book
7	POST	/api/books	Staff	Create book
8	PUT	/api/books/{id}	Staff	Update book
9	DEL	/api/books/{id}	Staff	Delete book
10	GET	/api/students	Staff	List students
11	GET	/api/students/{id}	Staff	Get student
12	POST	/api/students	Staff	Create student + account
13	PUT	/api/students/{id}	Staff	Update student
14	DEL	/api/students/{id}	Staff	Delete student
15	GET	/api/librarians	Admin	List librarians
16	GET	/api/librarians/{id}	Admin	Get librarian
17	POST	/api/librarians	Admin	Create librarian + account
18	PUT	/api/librarians/{id}	Admin	Update librarian
19	DEL	/api/librarians/{id}	Admin	Delete librarian
20	GET	/api/borrow-records	Staff	List borrow records
21	POST	/api/borrow-records	Staff	Borrow a book
22	POST	/api/borrow-records/{id}/return	Staff	Return a book
23	GET	/api/dashboard	Staff	Dashboard statistics
24	GET	/api/profile	Any	Current user profile

Table 11: Complete API endpoint reference. “Staff” = Admin or Librarian. “Any” = any authenticated user.

6 Application Workflows

6.1 Authentication Flow

1. The browser loads `index.html`, which initializes the JavaScript module graph.
2. The frontend calls `GET /api/auth/csrf` to obtain the CSRF token cookie.
3. It then calls `GET /api/auth/me` to check for an existing session (silently handles 401).
4. The user submits the login form → `POST /api/auth/login` with credentials and the CSRF header.
5. The backend authenticates via `DaoAuthenticationProvider` → BCrypt verification → creates a session.
6. The frontend stores the user identity and loads all data pages in parallel.
7. On logout, the session is invalidated server-side and the frontend returns to the login page.

6.2 Borrow Workflow

1. Staff opens the “Record a borrow” modal and fills in the Book ID, Student ID, and borrow date.
2. `POST /api/borrow-records` sends the request with CSRF protection.
3. The service validates that the book exists and is available, and that the student exists.
4. Borrower contact details are snapshotted from the student profile into the borrow record.
5. The book’s `available` flag is set to `false` within the same transaction.
6. The frontend displays a success toast and refreshes all data tables.

If the book is already borrowed, the system returns 400 Bad Request with code `BOOK_UNAVAILABLE`.

6.3 Return Workflow

1. Staff clicks the “Return” button on an active borrow record.
2. `POST /api/borrow-records/{id}/return` is sent with CSRF protection.
3. The service verifies the record exists and has not already been returned.
4. `return_date` is set to today’s date and the book’s `available` flag is restored to `true`.
5. The frontend shows a “Book returned” toast.

If the record was already returned, the system returns 400 Bad Request with code `ALREADY_RETURNED`.

7 Screenshots

7.1 Authentication

The screenshot shows the login page of the Library Management System. The page has a light blue background. At the top, there is a navigation bar with the 'Library Management' logo on the left and links for 'Dashboard', 'Books', 'Students', 'Librarians', 'Borrow records', 'Profile', and 'Logout' on the right. In the center, there is a white login card with a shadow. The card contains the text 'LIBRARY MANAGEMENT SYSTEM' and 'Welcome back'. Below this, it says 'Sign in to access the library workspace.' There are two input fields: 'Username' with the value 'admin' and 'Password' with masked characters. A blue 'Sign in' button is at the bottom of the card.

Figure 2: Login page — session-based authentication with CSRF protection.

7.2 Dashboard

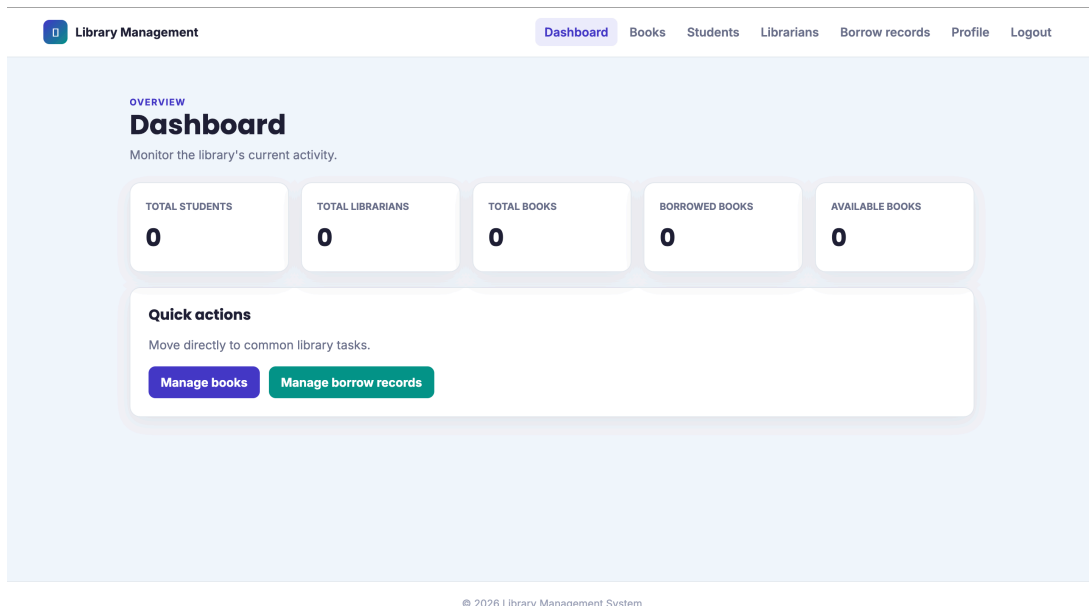


Figure 3: Admin dashboard showing aggregate statistics: total students, librarians, books, borrowed, and available.

7.3 Book Management

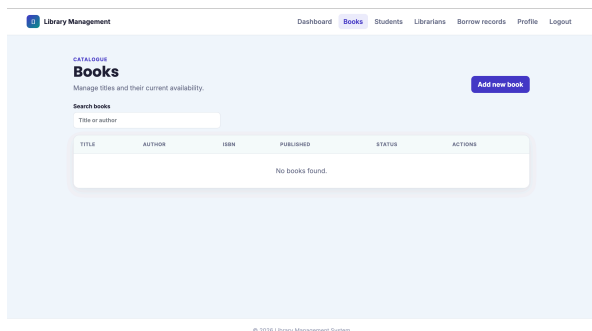


Figure 4: Books catalogue with search functionality.

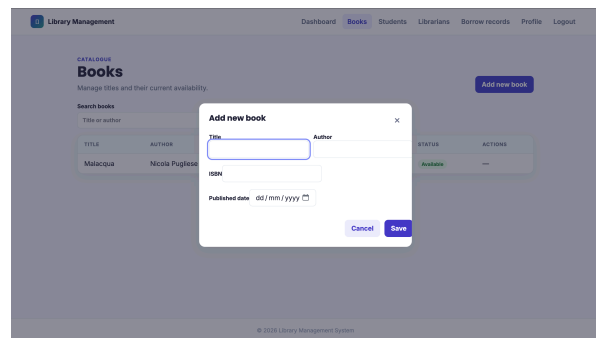


Figure 5: Add book modal with form validation.

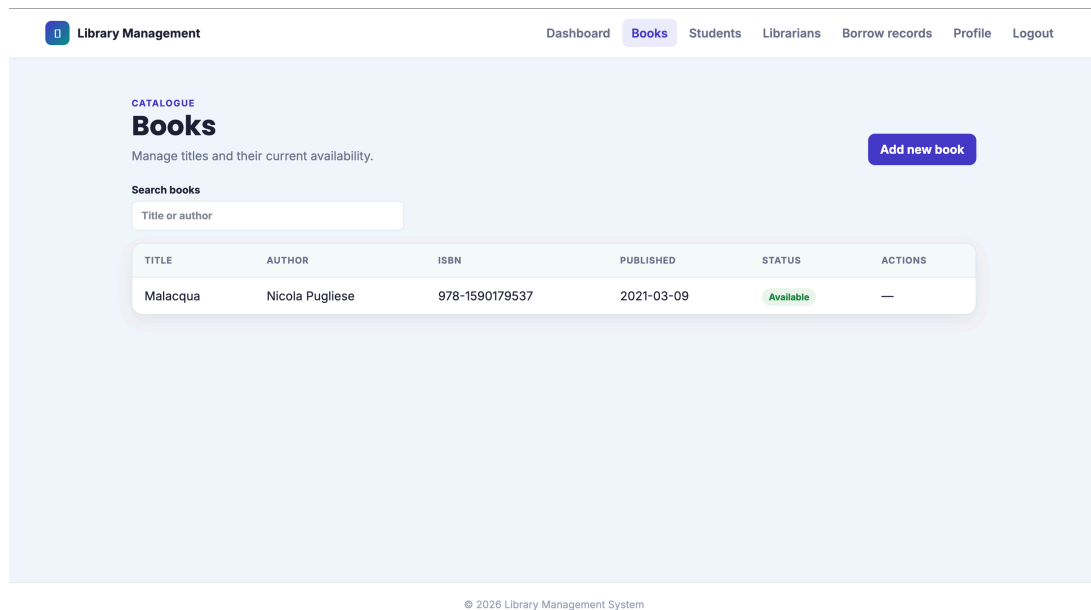


Figure 6: Book successfully added — shown with success toast notification.

7.4 Student and Librarian Management

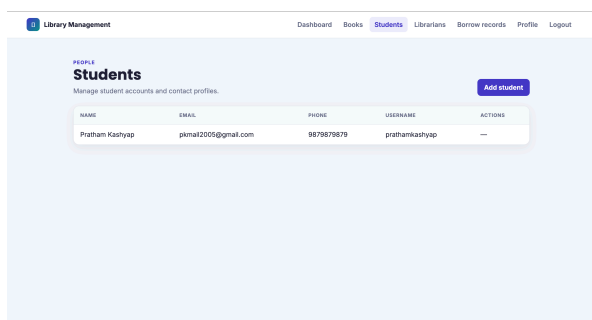


Figure 7: Students list view.

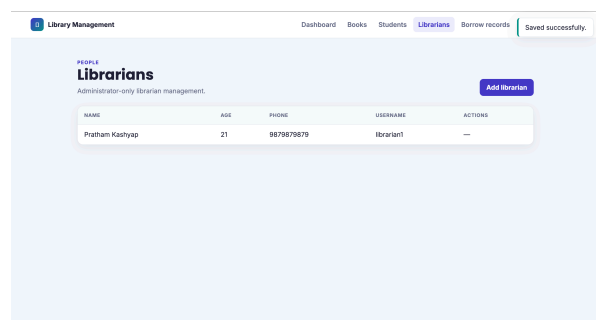


Figure 8: Librarians list view (admin-only).

Add student

Username is already in use.

Name: Pratham Kashyap

Email: pkmal2005@gmail.com

Phone: 9879879879

Username: prathamkashyap

Initial password: *****

Cancel Save

Figure 9: Add student modal.

Add librarian

Name: Pratham Kashyap

Age: 21

Phone: 9879879879

Username: prathamkashyap

Initial password: *****

Cancel Save

Figure 10: Add librarian modal.

7.5 Borrow Records

Record a borrow

Book ID: 1

Student ID: 1

Borrower name: Pratham Kashyap

Borrower email: pkmal2005@gmail.com

Borrower phone: 9879879879

Borrow date: 23/07/2026

Cancel Save

Figure 11: Record a borrow modal.

Borrow records

Track active and completed library loans.

BOOK	BORROWER	BORROWED	RETURNED	STATUS	ACTIONS
Malacca	Pratham Kashyap	2026-07-23	—	BORROWED	Return

Record a borrow

Saved successfully.

Figure 12: Borrow record saved with book marked as borrowed.

Borrow records

Track active and completed library loans.

BOOK	BORROWER	BORROWED	RETURNED	STATUS	ACTIONS
No borrow records found.					

Record a borrow

Figure 13: Borrow records table showing active and returned loans.

7.6 Profile

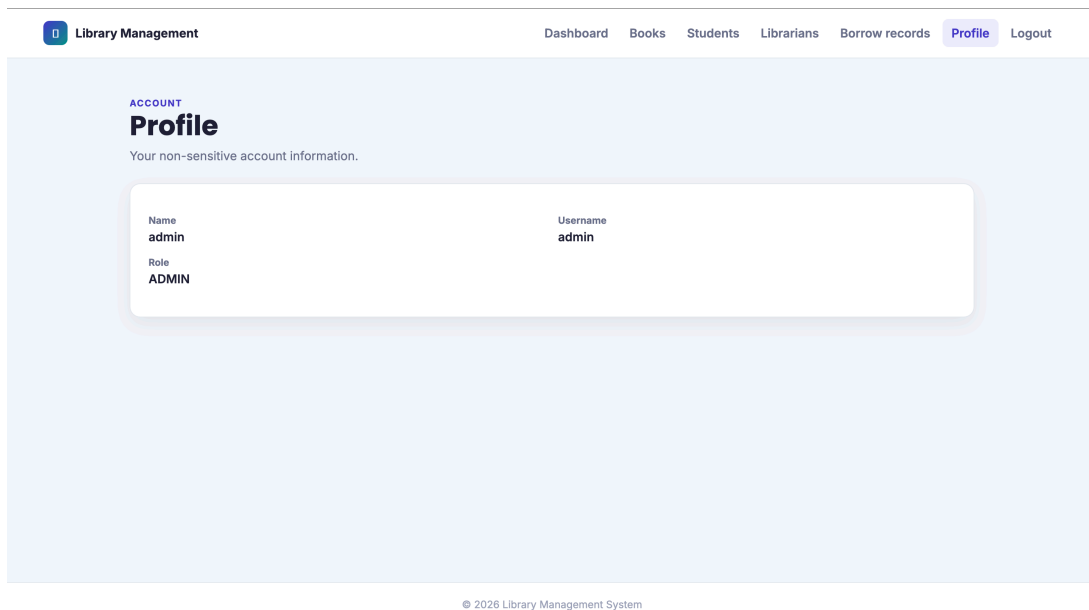


Figure 14: User profile page showing non-sensitive account information.

7.7 Validation and Error Handling

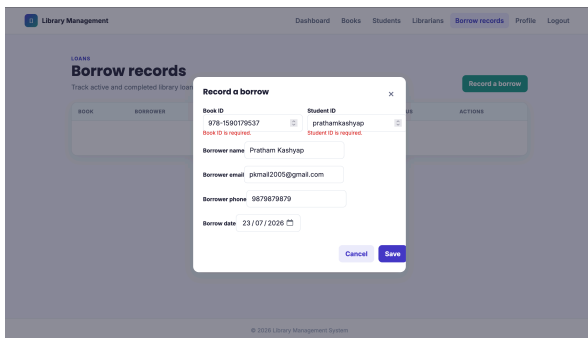


Figure 15: Server-side validation error rendering.

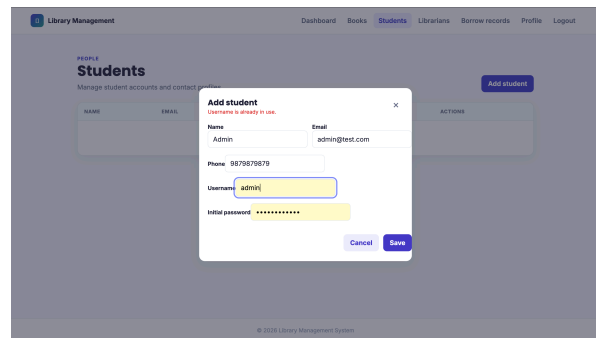


Figure 16: Duplicate username conflict (409).

7.8 Mobile Responsive Design

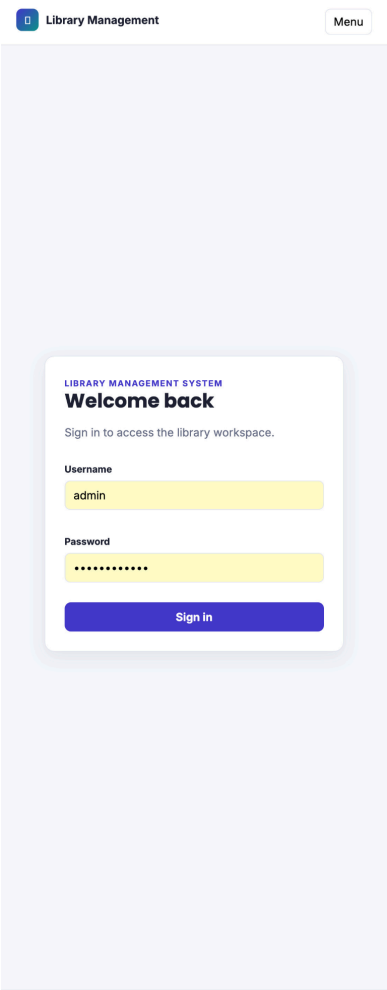


Figure 17: Mobile login.

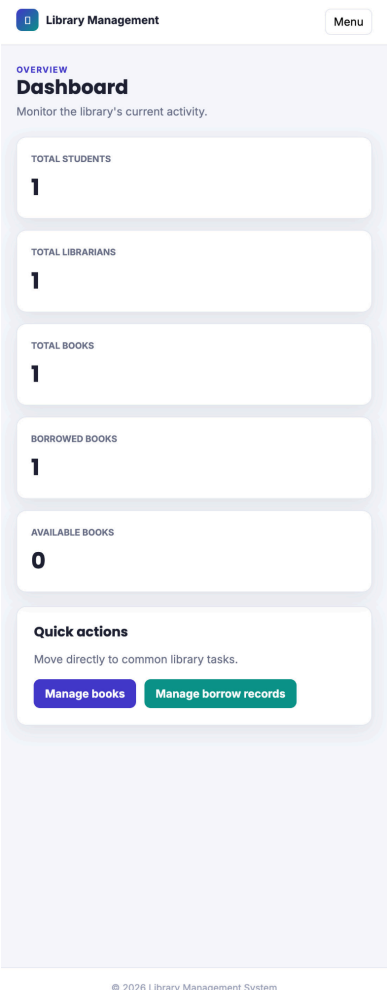


Figure 18: Mobile dashboard.

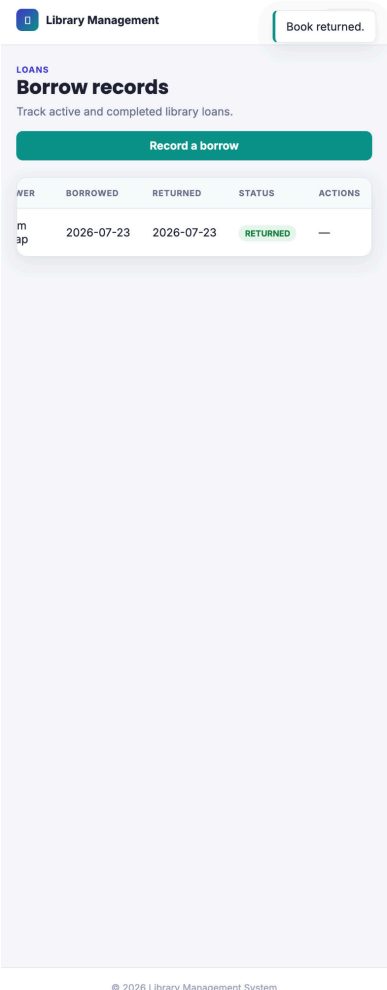


Figure 19: Mobile borrow return.

8 Testing

8.1 Test Infrastructure

The project uses JUnit 5 with Spring Boot Test and MockMvc for integration testing. Tests run against an H2 database in MySQL compatibility mode with create-drop schema management, ensuring a clean database for each test run.

8.2 Test Coverage

Test Class	Covers	Methods
LibraryManagementIntegrationTest	Authentication, CRUD lifecycle, borrow/return, validation errors, ISBN conflicts, role authorization (401/403), logout	4
BrowserCsrfFlowIntegrationTest	Full browser-equivalent CSRF flow: cookie bootstrap → header exchange → login → session reuse → logout → post-logout rejection	1
BookRepositoryTest	JPA auditing timestamp population, ISBN uniqueness constraint at database level	1

Table 12: Test classes and coverage areas.

8.3 Key Test Scenarios

1. **Full lifecycle test:** Creates a student, librarian, and book → borrows the book → verifies unavailability → returns it → verifies re-availability → confirms deletion conflict → logs out.
2. **Security test:** Verifies unauthenticated requests receive 401 Unauthorized and librarians accessing admin-only endpoints receive 403 Forbidden.
3. **CSRF regression test:** Performs the exact browser flow (cookie → header → login → session → logout) to verify the custom CSRF handler works correctly.
4. **Validation test:** Submits invalid book data and verifies field-level error responses.
5. **Conflict test:** Creates a book with a duplicate ISBN and verifies the 409 Conflict response.

8.4 Test Execution

All tests are executed with:

```
./mvnw clean test
```

The tests pass against the H2 in-memory database without requiring a running MySQL instance.

9 Future Scope

The following enhancements are planned for future releases:

1. **Pagination:** Add paginated result sets to all list endpoints to support large datasets.
2. **Update and Delete UI:** Build frontend forms and confirmation dialogs for the update and delete operations that already exist in the backend.
3. **Advanced Search:** Full-text search for books and filtering by availability, date range, and borrower for borrow records.
4. **Password Management:** Password-change endpoint and optional password-reset flow.
5. **Book Categories:** Categorization system for browsing and filtering by genre or subject.
6. **Fine System:** Overdue calculation and fine tracking for late returns.
7. **Notifications:** Email or in-app notifications for due dates and overdue books.
8. **Containerized Deployment:** Docker and Docker Compose setup for MySQL and the Spring Boot application.
9. **API Documentation:** Auto-generated Swagger/OpenAPI documentation.
10. **Database Migrations:** Introduction of Flyway or Liquibase for production-safe schema evolution.

10 Conclusion

The Library Management System was designed and implemented as a full-stack web application that demonstrates competency across the software development lifecycle. The project successfully delivers:

- A clean, four-layered MVC architecture with proper separation of concerns.
- A secure authentication and authorization system using Spring Security with BCrypt, sessions, and CSRF protection.
- A RESTful API with 24 endpoints, uniform error handling, and server-side validation.
- A responsive single-page application with real-time search, modal forms, and mobile-friendly design.
- A transactional borrow/return workflow that maintains data integrity and preserves audit history.
- Integration tests covering critical paths: authentication, authorization, CRUD operations, business rule enforcement, and CSRF flow.

The project provides a solid foundation for future enhancements including pagination, advanced search, deployment infrastructure, and expanded user-facing features. The architecture and code organization are designed to accommodate these additions without requiring significant restructuring.

11 References

1. Spring Boot Documentation, VMware Inc. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. Spring Security Reference, VMware Inc. <https://docs.spring.io/spring-security/reference/>
3. Spring Data JPA Reference, VMware Inc. <https://docs.spring.io/spring-data/jpa/reference/>
4. Jakarta Bean Validation Specification. <https://beanvalidation.org/>
5. MDN Web Docs — Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
6. OWASP CSRF Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
7. MySQL 8.0 Reference Manual, Oracle Corp. <https://dev.mysql.com/doc/refman/8.0/en/>